

| Universidad de Oriente |                                                  |
|------------------------|--------------------------------------------------|
| Machine Learning       | Clase 09 - Ejemplo 1 - Regresión lineal múltiple |

## Ejemplo 1

La base de datos contiene 20,000 registros estructurados de estudiantes que recopilan información relacionada con hábitos de estudio, patrones de uso digital, indicadores de bienestar, métricas de productividad y resultados académicos.

El conjunto de datos incluye variables como horas diarias de estudio, uso de teléfono inteligente y redes sociales, duración del sueño, niveles de estrés, puntuaciones de concentración, puntuaciones de productividad, asistencia y calificaciones finales. Permite un análisis integral de cómo diferentes factores conductuales y de estilo de vida influyen de manera conjunta en la productividad y el éxito académico de los estudiantes.

- Describir el conjunto de datos.
- Dividir el conjunto de datos en 25% de observaciones para la validación y el 75% restante para el entrenamiento y obtener el modelo de regresión lineal múltiple.
- Determinar cuáles variables no son significativas para el modelo.
- Obtener el  $R^2$  y el error cuadrático medio del modelo de regresión.
- Evaluar la posible multicolinealidad entre las variables predictoras mediante el Factor de Inflación de la Varianza (VIF) y los autovalores de la matriz de correlación.
- Realizar un gráfico de dispersión de los valores reales vs los valores predichos.
- Determinar el puntaje de productividad esperado tomando como valor la mediana de todas las variables predictoras

```
In [4]: #Cargar librerías
import pandas as pd
import os
import tarfile
import urllib.request
import numpy as np
from pandas.core.common import flatten
from plotnine import *
from array import *
import seaborn as sns
import scipy.stats as stats
import math
import matplotlib as mpl
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn import linear_model
from sklearn.metrics import mean_squared_error, r2_score
```

```
import statsmodels.formula.api as smf
import statsmodels.stats.api as sms
```

```
In [6]: #Cargar la base de datos
datos = pd.read_csv("student_productivity.csv")
```

Para describir el conjunto de datos se realiza un análisis exploratorio inicial utilizando los métodos `.info()` y `.describe()`, junto con histogramas y diagramas de caja y bigote.

Mediante `.info()` se identifica la estructura general de la base de datos, el número de registros, el tipo de datos de cada variable y la posible presencia de valores nulos.

Con `.describe()` se obtienen medidas estadísticas descriptivas como la media, desviación estándar, valores mínimos y máximos, así como los cuartiles, lo que permite conocer la tendencia central y la dispersión de las variables numéricas.

```
In [8]: datos.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   student_id                           20000 non-null  int64
1   age                                   20000 non-null  int64
2   gender                               20000 non-null  object
3   study_hours_per_day                  20000 non-null  float64
4   sleep_hours                          20000 non-null  float64
5   phone_usage_hours                    20000 non-null  float64
6   social_media_hours                   20000 non-null  float64
7   youtube_hours                        20000 non-null  float64
8   gaming_hours                         20000 non-null  float64
9   breaks_per_day                       20000 non-null  int64
10  coffee_intake_mg                     20000 non-null  int64
11  exercise_minutes                     20000 non-null  int64
12  assignments_completed                20000 non-null  int64
13  attendance_percentage                20000 non-null  float64
14  stress_level                         20000 non-null  int64
15  focus_score                          20000 non-null  int64
16  final_grade                          20000 non-null  float64
17  productivity_score                   20000 non-null  float64
dtypes: float64(9), int64(8), object(1)
memory usage: 2.7+ MB
```

```
In [10]: datos.describe()
```

Out[10]:

|       | student_id   | age          | study_hours_per_day | sleep_hours  | phone_usage_hours |
|-------|--------------|--------------|---------------------|--------------|-------------------|
| count | 20000.000000 | 20000.000000 | 20000.000000        | 20000.000000 | 20000.000000      |
| mean  | 10000.500000 | 23.00745     | 5.254562            | 6.517799     | 6.25047           |
| std   | 5773.647028  | 3.75489      | 2.742876            | 2.029784     | 3.31308           |
| min   | 1.000000     | 17.00000     | 0.500000            | 3.000000     | 0.50000           |
| 25%   | 5000.750000  | 20.00000     | 2.900000            | 4.770000     | 3.38000           |
| 50%   | 10000.500000 | 23.00000     | 5.250000            | 6.510000     | 6.24000           |
| 75%   | 15000.250000 | 26.00000     | 7.640000            | 8.310000     | 9.10250           |
| max   | 20000.000000 | 29.00000     | 10.000000           | 10.000000    | 12.00000          |

In [12]:

```
#Crear matriz de correlación
corr = datos.drop(["gender", "student_id"], axis=1).corr()
corr.style.background_gradient()
```

Out[12]:

|                       | age       | study_hours_per_day | sleep_hours | phone_usage_hours |
|-----------------------|-----------|---------------------|-------------|-------------------|
| age                   | 1.000000  | -0.005029           | 0.010650    | 0.009090          |
| study_hours_per_day   | -0.005029 | 1.000000            | 0.001670    | 0.011539          |
| sleep_hours           | 0.010650  | 0.001670            | 1.000000    | 0.000016          |
| phone_usage_hours     | 0.009090  | 0.011539            | 0.000016    | 1.000000          |
| social_media_hours    | 0.003258  | 0.004888            | -0.005939   | 0.004030          |
| youtube_hours         | 0.013934  | -0.002238           | 0.004920    | 0.004108          |
| gaming_hours          | 0.001777  | -0.002104           | -0.003666   | -0.007497         |
| breaks_per_day        | -0.006727 | -0.010371           | -0.000677   | -0.005431         |
| coffee_intake_mg      | 0.006629  | 0.004788            | 0.000882    | -0.000064         |
| exercise_minutes      | 0.001646  | 0.004380            | 0.006526    | -0.003186         |
| assignments_completed | -0.003458 | 0.005643            | 0.000944    | 0.006213          |
| attendance_percentage | -0.006531 | -0.008083           | 0.003433    | -0.002986         |
| stress_level          | -0.002612 | 0.001781            | 0.000020    | 0.006138          |
| focus_score           | -0.004130 | 0.004536            | -0.003241   | -0.000206         |
| final_grade           | -0.000857 | -0.012208           | 0.010543    | -0.012136         |
| productivity_score    | -0.005423 | 0.732777            | 0.340902    | -0.326650         |

In [14]:

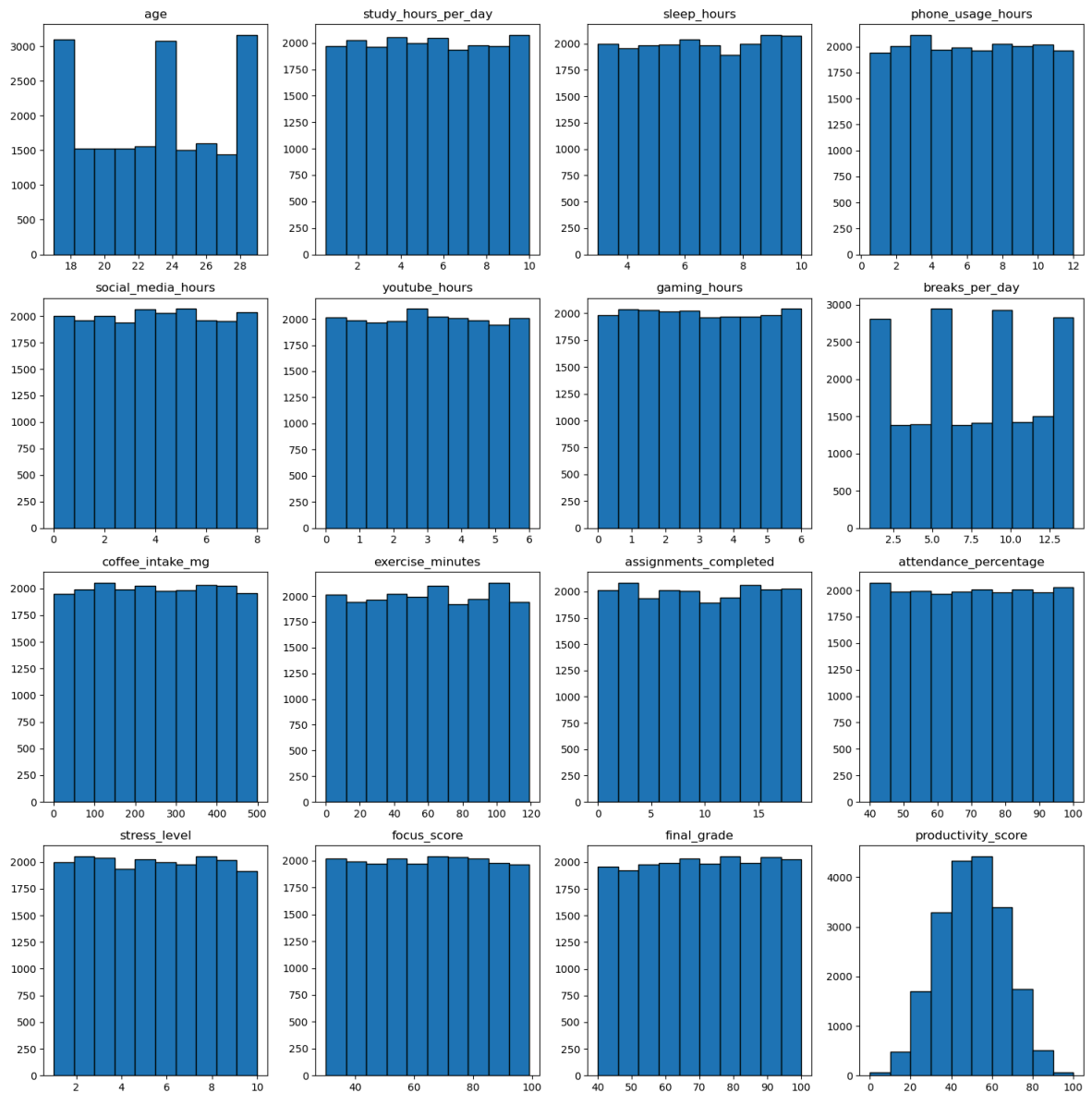
```
vars_num = datos.select_dtypes(include='number').columns.drop(["student_id"]).to_list()

plt.figure(figsize=(15,15))

for i, var in enumerate(vars_num):
```

```
plt.subplot(4, 4, i+1)
plt.hist(datos[var], edgecolor="black", bins=10)
plt.title(var)

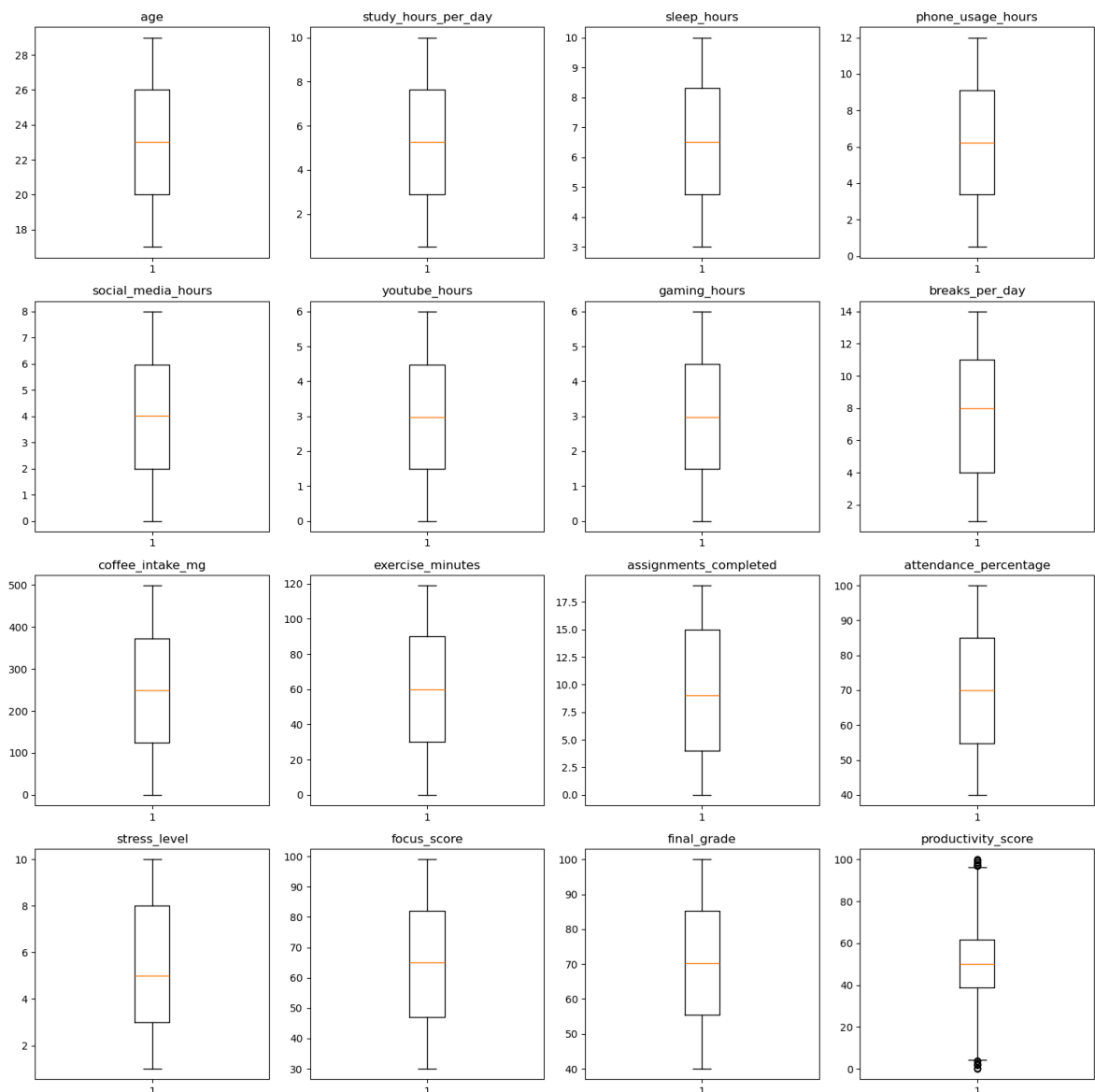
plt.tight_layout()
plt.show()
```



```
In [16]: #Diagramas de caja y bigote
plt.figure(figsize=(15,15))

for i, var in enumerate(vars_num):
    plt.subplot(4, 4, i+1)
    plt.boxplot(datos[var])
    plt.title(var)

plt.tight_layout()
plt.show()
```



A partir del método `.info()` se observa que el conjunto de datos está compuesto por 20,000 registros y 18 variables, incluyendo variables numéricas y una variable categórica. Asimismo, todas las variables presentan 20,000 valores no nulos, lo que indica que no existen datos faltantes en el conjunto, por lo que no es necesario realizar un procesamiento previo relacionado con la imputación o eliminación de valores ausentes.

Por otro lado, mediante los diagramas de caja y bigote se identifica la presencia de un dato atípico en la variable `productivity_score`, lo cual sugiere la existencia de un valor extremo que se aleja del comportamiento general de los datos. Este tipo de observaciones resulta importante, ya que los valores atípicos pueden influir en el análisis estadístico y en el desempeño de los modelos predictivos.

b) Se realiza la selección de las variables numéricas del conjunto de datos, excluyendo la variable identificadora `student_id` y la variable objetivo `productivity_score`, con el fin de definir las variables independientes del modelo. De esta forma, se construye la matriz de características  $X$  a partir de las variables explicativas, mientras que la variable `productivity_score` se define como la variable dependiente.

Posteriormente, se divide el conjunto de datos en subconjuntos de entrenamiento y prueba utilizando la función `train_test_split`. Se asigna el 75% de los datos para entrenamiento y el 25% para prueba, lo que permite entrenar el modelo y evaluar su desempeño en datos no utilizados durante el entrenamiento. Además, se establece una semilla aleatoria ( `random_state=2026` ) para garantizar la reproducibilidad de los resultados.

```
In [20]: from sklearn.model_selection import train_test_split

vars_num = datos.select_dtypes(include='number').columns.drop(["student_id", "pro

x_independientes = datos[vars_num]
y_dependiente = datos["productivity_score"].copy()

#Obtenemos las muestras de entrenamiento y test por medio de la función train_te
X_train, X_test, y_train, y_test= train_test_split(x_independientes, y_dependien
```

```
In [22]: #Crear la regresión
from sklearn.linear_model import LinearRegression

lm = LinearRegression()
lm.fit(X_train, y_train)
```

```
Out[22]: ▼ LinearRegression ⓘ ?
LinearRegression()
```

```
In [24]: #Se muestra el intercepto
intercepto = lm.intercept_
print("El intercepto es que se tiene es %s" %intercepto)
```

El intercepto es que se tiene es -6.238752173705365

```
In [26]: #Obtención de los coeficientes
coeficientes = lm.coef_

#Crear dataframe para mostrar los coeficientes
pd.DataFrame({"Variables": x_independientes.columns, "Coeficientes": coeficiente
```

Out[26]:

|    | Variables             | Coeficientes  |
|----|-----------------------|---------------|
| 0  | age                   | 7.176968e-07  |
| 1  | study_hours_per_day   | 4.316388e+00  |
| 2  | sleep_hours           | 2.697745e+00  |
| 3  | phone_usage_hours     | -1.618631e+00 |
| 4  | social_media_hours    | -5.237254e-06 |
| 5  | youtube_hours         | 1.198278e-05  |
| 6  | gaming_hours          | 1.943411e-05  |
| 7  | breaks_per_day        | -3.408801e-06 |
| 8  | coffee_intake_mg      | 1.358410e-07  |
| 9  | exercise_minutes      | 8.412566e-07  |
| 10 | assignments_completed | -7.186053e-06 |
| 11 | attendance_percentage | 1.618638e-01  |
| 12 | stress_level          | -1.079096e+00 |
| 13 | focus_score           | 3.237287e-01  |
| 14 | final_grade           | -1.719381e-06 |

c) Se construye un modelo de regresión lineal múltiple utilizando la librería `statsmodels`. En primer lugar, se añade el término independiente a las variables explicativas mediante la función `add_constant`, lo cual es necesario para estimar correctamente el modelo.

Posteriormente, se ajusta el modelo de regresión lineal utilizando las variables de entrenamiento. Una vez ajustado el modelo, se obtiene un resumen estadístico mediante `summary()`, el cual proporciona información relevante como coeficientes estimados, errores estándar, valores t y valores p.

Este resumen se utiliza para evaluar la significancia estadística de cada variable independiente. En particular, los valores p permiten identificar aquellas variables que no son significativas dentro del modelo, es decir, aquellas cuyo valor p es mayor que un nivel de significancia comúnmente 0.05, indicando que no contribuyen de manera relevante a la explicación de la variable dependiente.

```
In [28]: import statsmodels.api as sm
from scipy import stats

#Añadir el intercepto
X2 = sm.add_constant(X_train)
#Ajustar el modelo
est = sm.OLS(y_train, X2)

#Ver el ajuste
```

```
est2 = est.fit()  
print(est2.summary())
```



## OLS Regression Results

```

=====
Dep. Variable:      productivity_score      R-squared:                1.000
Model:              OLS                    Adj. R-squared:           1.000
Method:             Least Squares          F-statistic:             3.138e+10
Date:               Fri, 20 Mar 2026        Prob (F-statistic):       0.00
Time:               10:21:24               Log-Likelihood:           66500.
No. Observations:   15000                 AIC:                     -1.330e+05
Df Residuals:       14984                 BIC:                     -1.328e+05
Df Model:           15
Covariance Type:    nonrobust
=====
=====
                                coef      std err          t      P>|t|      [0.025
0.975]
-----
const                -6.2388         0.000   -2.39e+04    0.000    -6.239
-6.238
age                   7.177e-07    6.28e-06     0.114    0.909   -1.16e-05
1.3e-05
study_hours_per_day   4.3164    8.56e-06    5.04e+05    0.000     4.316
4.316
sleep_hours           2.6977    1.16e-05    2.33e+05    0.000     2.698
2.698
phone_usage_hours     -1.6186    7.08e-06   -2.29e+05    0.000    -1.619
-1.619
social_media_hours    -5.237e-06  1.02e-05    -0.513    0.608   -2.53e-05
1.48e-05
youtube_hours         1.198e-05    1.36e-05     0.882    0.378   -1.47e-05
3.86e-05
gaming_hours          1.943e-05    1.36e-05     1.430    0.153   -7.21e-06
4.61e-05
breaks_per_day        -3.409e-06  5.87e-06    -0.581    0.561   -1.49e-05
8.09e-06
coffee_intake_mg      1.358e-07    1.63e-07     0.833    0.405   -1.84e-07
4.55e-07
exercise_minutes      8.413e-07    6.8e-07     1.238    0.216   -4.91e-07
2.17e-06
assignments_completed -7.186e-06  4.05e-06    -1.775    0.076   -1.51e-05
7.51e-07
attendance_percentage  0.1619    1.35e-06    1.2e+05    0.000     0.162
0.162
stress_level          -1.0791    8.19e-06   -1.32e+05    0.000    -1.079
-1.079
focus_score           0.3237    1.16e-06    2.78e+05    0.000     0.324
0.324
final_grade           -1.719e-06  1.36e-06    -1.265    0.206   -4.38e-06
9.45e-07
=====
Omnibus:              12672.824    Durbin-Watson:           1.995
Prob(Omnibus):         0.000    Jarque-Bera (JB):        894.375
Skew:                  0.011    Prob(JB):                6.15e-195
Kurtosis:              1.804    Cond. No.                 3.47e+03
=====

```

## Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 3.47e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Se observa que las variables age, social\_media\_hours, youtube\_hours, gaming\_hours, breaks\_per\_day, coffee\_intake\_mg, exercise\_minutes, assignments\_completed y final\_grade presentan valores p mayores a 0.05, por lo que no son estadísticamente significativas dentro del modelo. Esto indica que dichas variables no contribuyen de manera relevante a explicar la variabilidad de la variable dependiente productivity\_score en este ajuste del modelo.

d) Con el fin de cuantificar el modelo de regresión creado se determina el coeficiente de determinación  $R^2$  y el error cuadrático medio.

```
In [32]: #Obtener predicción para X_test
y_pred = lm.predict(X_test)

#Error cuadrático medio (MSE)
mse = mean_squared_error(y_test, y_pred)
#Coeficiente de determinación
r2 = r2_score(y_test, y_pred)

print("El error cuadrático medio que se tiene es %s" %mse)
print("El coeficiente de determinación del modelo es %.10f" %r2)
```

El error cuadrático medio que se tiene es 8.140432937370768e-06

El coeficiente de determinación del modelo es 0.9999999683

El modelo presenta un error cuadrático medio (MSE) de  $8.140432937370768 \times 10^{-6}$ , lo cual indica que, en promedio, las diferencias entre los valores reales y los valores predichos son extremadamente pequeñas, reflejando un ajuste muy preciso del modelo a los datos.

El coeficiente de determinación indica que el modelo explica prácticamente el 100% de la variabilidad de la variable dependiente. Esto sugiere un ajuste casi perfecto.

Se evalúa la presencia de multicolinealidad entre las variables independientes mediante el cálculo del Factor de Inflación de la Varianza (VIF). Para ello, se construye un DataFrame que contiene cada una de las variables explicativas y su correspondiente valor de VIF, utilizando la función `variance_inflation_factor` de la librería `statsmodels`.

El VIF permite medir cuánto se incrementa la varianza de los coeficientes estimados debido a la correlación entre las variables independientes. Valores de VIF cercanos a 1 indican baja correlación, mientras que valores superiores a 5 o 10 sugieren problemas de multicolinealidad que pueden afectar la estabilidad e interpretación del modelo. Este análisis es fundamental para validar la independencia entre las variables explicativas antes de ajustar el modelo de regresión.

```
In [34]: # Variance Inflation Factor
from statsmodels.stats.outliers_influence import variance_inflation_factor

#Crear dataframe para aplicar VIF
VIF = pd.DataFrame()
```

```
VIF["variable"] = x_independientes.columns

#Aplicar el método
VIF["VIF"] = [variance_inflation_factor(x_independientes.values, i) for i in range(x_independientes.shape[0])]

VIF
```

Out[34]:

|    | variable              | VIF       |
|----|-----------------------|-----------|
| 0  | age                   | 26.843143 |
| 1  | study_hours_per_day   | 4.523880  |
| 2  | sleep_hours           | 10.434517 |
| 3  | phone_usage_hours     | 4.432894  |
| 4  | social_media_hours    | 3.923295  |
| 5  | youtube_hours         | 3.911672  |
| 6  | gaming_hours          | 3.884712  |
| 7  | breaks_per_day        | 4.386410  |
| 8  | coffee_intake_mg      | 3.939914  |
| 9  | exercise_minutes      | 3.877844  |
| 10 | assignments_completed | 3.613201  |
| 11 | attendance_percentage | 14.944622 |
| 12 | stress_level          | 4.509780  |
| 13 | focus_score           | 10.281771 |
| 14 | final_grade           | 15.156632 |

A partir de los resultados obtenidos del Factor de Inflación de la Varianza (VIF), se observa que algunas variables presentan valores elevados, lo que indica la presencia de multicolinealidad en el modelo. En particular, las variables age , attendance\_percentage , final\_grade, sleep\_hours y focus\_score superan el umbral de 10, lo que sugiere una alta correlación con otras variables independientes y posibles problemas en la estimación de los coeficientes.

Por otro lado, el resto de variables presentan valores de VIF entre 3 y 5, lo cual indica una multicolinealidad moderada, pero generalmente aceptable dentro del modelo. En conjunto, estos resultados evidencian que existe dependencia lineal entre algunas variables explicativas, por lo que sería recomendable considerar la eliminación o transformación de aquellas con VIF elevado para mejorar la estabilidad e interpretación del modelo de regresión.

```
In [36]: #Obtención de los autovalores de la matriz de correlación
corr_ind = x_independientes.corr()
av, v = np.linalg.eig(corr_ind)
AV = pd.DataFrame()
AV["Variable"] = x_independientes.columns
AV["Autovalor"] = av
```

AV

Out[36]:

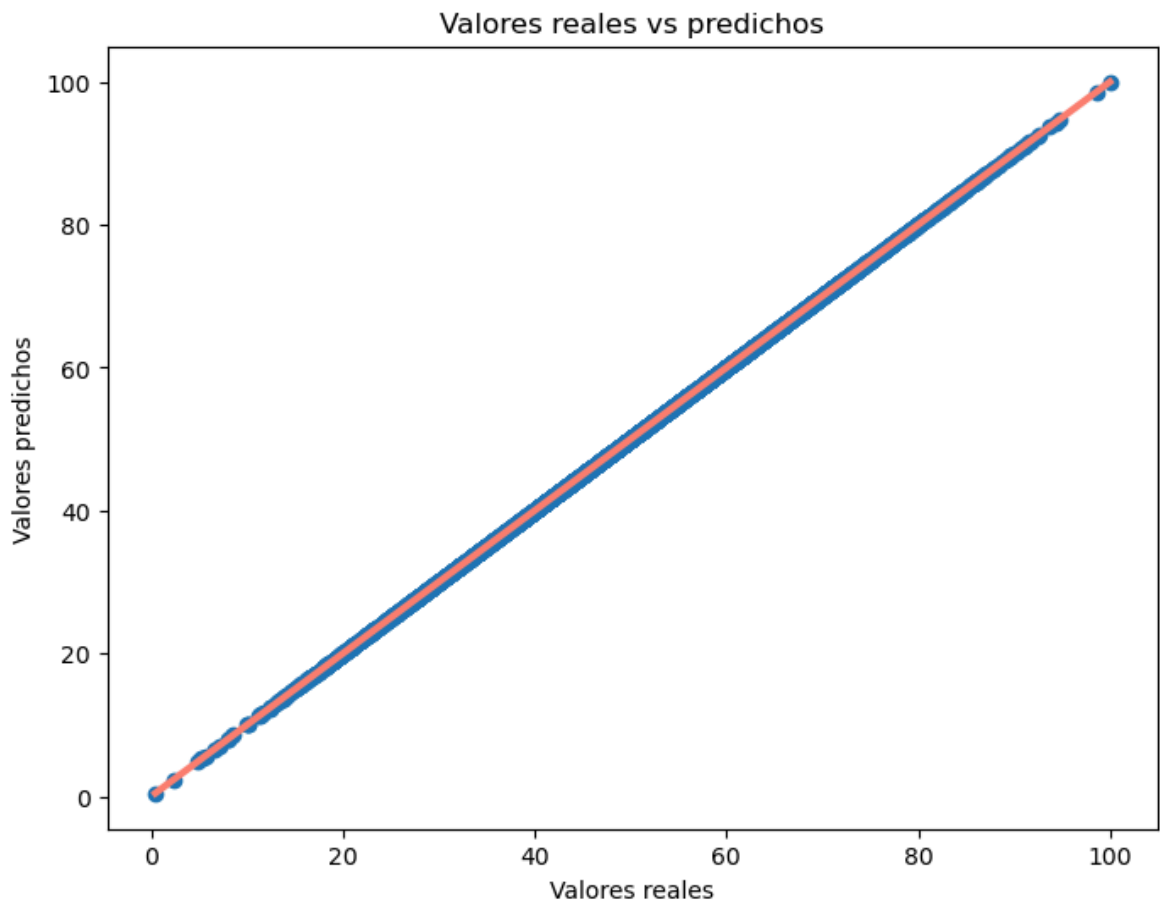
|    | Variable              | Autovalor |
|----|-----------------------|-----------|
| 0  | age                   | 0.957475  |
| 1  | study_hours_per_day   | 1.040345  |
| 2  | sleep_hours           | 1.038775  |
| 3  | phone_usage_hours     | 0.963613  |
| 4  | social_media_hours    | 0.968633  |
| 5  | youtube_hours         | 1.029264  |
| 6  | gaming_hours          | 1.019720  |
| 7  | breaks_per_day        | 1.016257  |
| 8  | coffee_intake_mg      | 1.012196  |
| 9  | exercise_minutes      | 0.980772  |
| 10 | assignments_completed | 0.985684  |
| 11 | attendance_percentage | 0.990514  |
| 12 | stress_level          | 0.995189  |
| 13 | focus_score           | 0.998709  |
| 14 | final_grade           | 1.002856  |

A partir de los autovalores de la matriz de correlación, se observa que todos los valores son cercanos a 1 y ninguno se aproxima a 0. Esto indica que no existen combinaciones lineales fuertes entre las variables independientes, lo que sugiere que no hay problemas severos de multicolinealidad desde la perspectiva espectral.

En general, autovalores muy pequeños cercanos a 0 evidencian la presencia de dependencia lineal entre variables; sin embargo, en este caso los autovalores se mantienen en un rango aproximadamente entre 0.95 y 1.04. Por tanto, el conjunto de variables presenta una adecuada independencia lineal global, aunque algunos indicadores como el VIF pueden señalar relaciones particulares entre ciertas variables que conviene analizar de manera individual.

f) Se realiza el gráfico de dispersión para comparar los valores reales y los predichos, para corroborar los resultados del  $R^2$ . Al ser un  $R^2$  cercano a uno, se espera que los datos se ajusten perfectamente a la recta identidad.

```
In [39]: plt.figure(figsize=(8,6))
plt.scatter(y_test,y_pred)
plt.xlabel("Valores reales")
plt.ylabel("Valores predichos")
plt.title("Valores reales vs predichos")
plt.plot([min(y_test),max(y_test)],[min(y_test),max(y_test)],color='salmon',line
plt.show()
```



g) Para estimar el puntaje de productividad esperado, se consideran como valores de entrada las medianas de todas las variables predictoras. Esto implica construir un vector representativo del estudiante típico, donde cada variable independiente toma su valor mediano

```
In [45]: #Se obtiene la mediana y se transforma en array
medianas = np.array(np.median(x_independientes,axis=0))

#Predicción
prediccion_medianas = lm.predict(medianas.reshape(1,-1))

print("El puntaje de productividad esperado es de %s" %prediccion_medianas)
```

El puntaje de productividad esperado es de [50.85922925]

C:\Users\USER\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names

## Fin

In [ ]: